

Zig Project Architecture Standard

Atlas Architecture Group

August 17, 2026

Contents

0.1	0. Standard Positioning	2
0.1.1	0.1 Purpose	2
0.2	1. Authority Model	3
0.3	2. Versioning Boundary	3
0.4	3. Core Thesis —Explicit Architecture	4
0.5	4. Zig Architecture Ontology	5
0.6	5. Seven Independent Architecture Views	5
0.6.1	5.1 Physical Source Tree	5
0.6.2	5.2 Semantic Container / Namespace View	5
0.6.3	5.3 Module Dependency Graph	6
0.6.4	5.4 Declaration Discovery Graph	6
0.6.5	5.5 Package Dependency / Lifecycle View	6
0.6.6	5.6 Artifact / Product Graph	7
0.6.7	5.7 Build Step DAG	7
0.7	6. Fundamental Distinction	7
0.8	7. The Eight Architecture Laws	8
0.8.1	ZIG-ARCH-01 —Source Files Are Semantic Containers	8
0.8.2	ZIG-ARCH-02 —Filesystem Hierarchy Is Not the Dependency Graph	8
0.8.3	ZIG-ARCH-03 —Modules Are Explicit Compilation Boundaries	9
0.8.4	ZIG-ARCH-04 —Packages Represent Independent Lifecycle Boundaries	9
0.8.5	ZIG-ARCH-05 —Artifacts Are Orthogonal to Modules	10
0.8.6	ZIG-ARCH-06 —Discovery Must Be Intentional	10
0.8.7	ZIG-ARCH-07 —Structure Follows Semantic Pressure	10
0.8.8	ZIG-ARCH-08 —Naming Must Reveal Semantics	10
0.9	8. Boundary Decision Framework	11
0.9.1	8.1 When to Create a New File?	11
0.9.2	8.2 When to Create a Directory?	11
0.9.3	8.3 When to Create a Module?	12
0.9.4	8.4 When to Create a Package?	12
0.9.5	8.5 When to Create an Artifact?	12
0.9.6	8.6 When to Create a Build Step?	13
0.10	9. Source Organization Standard	13
0.10.1	9.1 File Naming	13
0.10.2	9.2 File-as-Type	13
0.11	10. Import Architecture	14
0.11.1	10.1 Local Implementation Relationship	14
0.11.2	10.2 Cross-Boundary Relationship	14
0.12	11. __INLINE_CODE_0__ Standard	14
0.12.1	11.1 Root Source File ≠ __INLINE_CODE_0__	14

0.12.2	11.2 Module Root as Intentional Public Surface	15
0.13	12. Process Boundary Standard	15
0.13.1	12.1 Thin <code>__INLINE_CODE_0__</code>	15
0.13.2	12.2 Zig 0.16 Process Capability Model	16
0.14	13. Public API Standard	16
0.15	14. Dependency Architecture Standard	17
0.15.1	14.1 Dependency Direction Must Be Explainable	17
0.15.2	14.2 Dependency Cycles	17
0.15.3	14.3 Deliberate Root Backreferences	18
0.16	15. Build Architecture Standard	18
0.16.1	15.1 <code>__INLINE_CODE_0__</code> Owns Build Topology	18
0.16.2	15.2 Permitted Build Work	19
0.16.3	15.3 Runtime Boundary	19
0.17	16. Test Architecture Standard	20
0.17.1	16.1 Unit Tests Near Semantics	20
0.17.2	16.2 Integration Tests Use Explicit Test Roots	20
0.17.3	16.3 Test Discovery Must Be Explicit	20
0.17.4	16.4 Dependency Module Tests Are Not Automatically the Root Suite	20
0.17.5	16.5 Test Compile $\neq$ Test Run	21
0.17.6	16.6 <code>__INLINE_CODE_0__</code> Is Not the Architecture Baseline	21
0.18	17. Naming and Semantic Cohesion Standard	21
0.18.1	17.1 Generic Names Require Evidence	21
0.18.2	17.2 Avoid Ceremonial Layers	21
0.19	18. Evolution Standard —Structure Must Be Earned	22
0.20	19. Reference Pattern A —Minimal Program	22
0.21	20. Reference Pattern B —Small Application	22
0.22	21. Reference Pattern C —Medium Application	22
0.23	22. Reference Pattern D —Multi-Artifact System	23
0.24	23. Reference Pattern E —Multi-Package Repository	24
0.25	24. Consolidated Anti-Patterns	24
0.25.1	AP-01 —Filesystem as Dependency Injection	24
0.25.2	AP-02 —Module per Folder	24
0.25.3	AP-03 —Package per Module	24
0.25.4	AP-04 —Fat <code>__INLINE_CODE_0__</code>	25
0.25.5	AP-05 —God <code>__INLINE_CODE_0__</code>	25
0.25.6	AP-06 —Runtime Control Plane in <code>__INLINE_CODE_0__</code>	25
0.25.7	AP-07 —Forced Flatness	25
0.25.8	AP-08 —Ceremonial Architecture	25
0.25.9	AP-09 —Semantic Garbage Buckets	25
0.25.10	AP-10 —Premature Shared Module	25
0.26	25. Architecture Decision Matrix	25
0.27	26. Conformance Review	26
0.27.1	Semantic Ownership	26
0.27.2	Dependency	26
0.27.3	Lifecycle	26
0.27.4	Product	26
0.27.5	Discovery & Testing	26
0.27.6	Build	26
0.27.7	API	26
0.27.8	Naming	26
0.28	27. Conformance Levels	27

0.28.1 Required	27
0.28.2 Recommended	27
0.28.3 Preference	27
0.29 28. Official Semantic Traceability	27
0.30 29. Frozen Engineering Decisions	27
0.31 30. Final Architecture Model	27
0.32 31. Final Manifesto	29
0.33 32. Architecture Freeze Statement	29

Version: 1.0

Status: Frozen Baseline

Language Baseline: Zig 0.16 series, 语义核验基于 Zig 0.16.0

Verified: 2026-08-14

Scope: Zig Application / CLI / Library / Systems Tool / Multi-Artifact Project / Multi-Package Repository

Purpose: Source Organization, Semantic Boundaries, Module Architecture, Package Lifecycle, Artifact Topology, Build Graph, Testing, Naming & Evolution

0.1 0. Standard Positioning

0.1.1 0.1 Purpose

本标准不是为了定义一套“唯一正确”的 Zig 项目目录模板。

Zig 官方定义的是语言与工具链机制，例如：

- Source File Semantics;
- Compilation Model;
- Module;
- root module;
- `__INLINE_CODE_0__`;
- File / Declaration Discovery;
- Build System;
- Package Management;
- Naming Style;
- Testing Model。

在这些机制之上，项目仍然必须作出大量 Architecture Decision。

因此，本标准的目标是建立一套：

Zig-native Architecture Reasoning Model

使项目能够明确回答：

- 一个 declaration 为什么属于这里；
- 一个 file 为什么应该存在；
- 一个 directory 表达什么语义；
- 一个 dependency 为什么跨越 Module；
- 一个 Module 为什么值得成为独立 boundary；
- 一个 Package 为什么具有独立生命周期；
- 一个 Artifact 为什么需要独立构建；
- 哪些代码实际上会进入 compilation；

- 哪些 tests 实际上会被发现和执行；
- `__INLINE_CODE_0__` 究竟拥有哪部分系统拓扑。

本标准冻结的是：

Architecture reasoning rules

而不是某一棵固定目录树。

0.2 1. Authority Model

本标准严格区分 Zig 官方事实与项目工程规则。

标记	Authority	定义
[O-S]	Official Semantics	Zig 官方语言、编译器、标准库或 Build System 明确定义的语义
[O-G]	Official Guidance	Zig 官方 Style Guide / Documentation 给出的指导性建议
[E]	Engineering Standard	基于 Zig 机制推导出的本标准工程规则
[P]	Project Preference	推荐默认值或项目结构偏好，不代表 Zig 普遍要求

本标准中的：

- **MUST**
- **MUST NOT**
- **SHOULD**
- **SHOULD NOT**
- **MAY**

仅表达：

采用本 **Architecture Standard** 后的工程约束强度。

它们不意味着 Zig 编译器本身强制要求这些结构。这是本标准最重要的认识论边界。

0.3 2. Versioning Boundary

Zig 尚未进入 1.0，语言、标准库、Build System 与 Package Management 仍可能发生较大变化。截至本标准核验日期，Zig 官方将 **0.16.0** 列为最新稳定版本。因此：

本标准的 [O-S] 与 [O-G] 内容 **MUST** 与明确的 **Zig Toolchain Baseline** 绑定。

当项目升级 Zig baseline 时：

- `__INLINE_CODE_0__` **MUST** 重新核验；
- `__INLINE_CODE_0__` **MUST** 重新核验；

- `__INLINE_CODE_0__` SHOULD 检查其原始语义依据是否仍然成立；
- `__INLINE_CODE_0__` 不因 Zig 版本变化自动获得或失去权威性。

换言之：



Architecture Standard **MUST NOT** 将特定版本偶然存在的 API 误写成永恒的 Zig 架构真理。

0.4 3. Core Thesis —Explicit Architecture

Zig 项目架构不应该被简化为：

Flat Architecture

也不应该被简化为：

Layered Architecture

本标准采用的核心原则是：

Explicit Architecture

结构必须来源于真实的软件语义和依赖压力，而不是来源于预制框架模板。

- 当 declarations 共同表达一个紧密语义能力时：**keep them together**
- 当一个 type 已经形成独立、复杂、可理解的行为边界时：**give it a coherent source file**
- 当两个子系统之间存在真正的 compilation / dependency boundary 时：**introduce a Module**
- 当一个组件拥有独立 dependency、identity、distribution 或 release lifecycle 时：**consider a Package**
- 当系统需要产生一个独立执行、链接、测试或工具产品时：**introduce an Artifact**

因此：

Structure follows semantic pressure.

而不是：

Structure follows framework convention.

0.5 4. Zig Architecture Ontology

Zig 项目不能通过一棵目录树完整描述。必须首先区分：

```
File
Directory
Module
Package
Artifact
Build Step
```

这些对象不是同一个抽象层。本标准禁止建立如下机械映射：

```
Directory = Module
Module    = Package
Package   = Artifact
```

也不采用这样的伪线性层级模型：

```
Declaration
  ↓
File
  ↓
Module
  ↓
Package
```

更准确的架构模型是多个相互关联、但彼此正交的视图。

0.6 5. Seven Independent Architecture Views

成熟 Zig 项目 SHOULD 能够同时解释以下七张图。

0.6.1 5.1 Physical Source Tree

回答：源码物理上放在哪里？

例如：

```
src/
├─ cli/
├─ engine/
├─ protocol/
└─ platform/
```

它表达 **Physical Organization**，而不是 **Dependency Graph**。Directory 首先解决的是定位、聚类与认知导航问题。

0.6.2 5.2 Semantic Container / Namespace View

回答：哪些 **declarations** 属于同一个语义 **container**？

Zig 0.16 明确定义：每个 Zig source file 隐式都是一个 `__INLINE_CODE_0__` declaration，文件顶层可以包含 declarations，也可以包含 fields；拥有 fields 的 source file 可以直接作为可实例化 type。

因此，`__INLINE_CODE_0__` 并不只是“装代码的文本文件”。它本身拥有语言级 container 语义。

一个 source file 可以自然表达：

- coherent namespace；

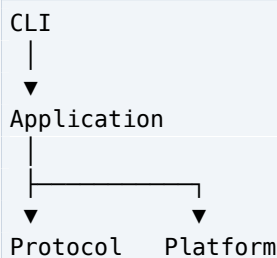
- coherent type;
- type + closely related operations;
- domain contract;
- implementation-local semantic cluster。

0.6.3 5.3 Module Dependency Graph

回答：哪个 **compilation boundary** 可以依赖哪个 **boundary**？

Zig Compilation 被划分为 Modules。每个 Module 是一组 source files，其中一个是 root source file；Module 可以依赖其他 Module，并通过 module name 使用 `__INLINE_CODE_0__` 导入其 root source file。官方同时明确允许 Module Dependency Graph 出现 dependency loops。

例如：



这是 **Compilation Dependency Architecture**，而不是目录树。

0.6.4 5.4 Declaration Discovery Graph

回答：编译器实际上会发现并分析哪些源码和 **declarations**？

Zig 不会简单递归扫描目录中的全部 `__INLINE_CODE_0__` 文件。0.16 Compilation Model 明确定义 discovery rules，包括：

- 被分析的 `__INLINE_CODE_0__` 会发现目标 file；
- 被分析的 type/file 会进一步发现其中相关 `__INLINE_CODE_0__` / `__INLINE_CODE_1__`；
- test compilation 对 root module 中被分析 type/file 的 test declarations 有额外 discovery 行为；
- 被使用的 named declaration 才进一步进入 semantic analysis。

因此：

File exists	≠	File is discovered
Declaration exists	≠	Declaration is analyzed
Test exists	≠	Test is necessarily part of this test compilation

Discovery 是 Architecture Reality 的一部分。

0.6.5 5.5 Package Dependency / Lifecycle View

回答：哪些组件拥有独立 **dependency identity** 与 **distribution lifecycle**？

Zig Package Management 使用 `__INLINE_CODE_0__` 等机制描述 package metadata 和 dependencies；0.16 package management 继续强化了 package identity，例如 dependency override 会涉及 package `__INLINE_CODE_1__` 与 `__INLINE_CODE_2__`。

但是，**Package = independent lifecycle boundary** 是本标准的工程解释 [E]，而不是对 Zig 官方 Package 概念的穷尽性定义。

本标准用 Package 表达：

- dependency lifecycle;
- identity;
- external consumption;
- independent distribution;
- independent version/release concerns;
- independent build ownership。

Package **SHOULD NOT** 只是“更大的目录”。

0.6.6 5.6 Artifact / Product Graph

回答：最终构建什么？

例如：



Zig Build System 可以构建 executable、static/dynamic library、test compilation 等产品，并将它们放入更大的 build topology。

因此：

Artifact Graph MUST NOT be mechanically derived from Module Graph.

同一个 Module 可以服务多个 Artifacts。一个 Artifact 也可以组合多个 Modules。

0.6.7 5.7 Build Step DAG

回答：构建任务之间存在什么 **dependency constraints**？

Zig Build System 将项目表示为由 Steps 构成的 Directed Acyclic Graph；无依赖的步骤可以独立、并发运行。

所以：

Module Dependency Graph $\neq$ Build Step DAG

尤其重要的是：__INLINE_CODE_0__ 官方允许 cycle；而 __INLINE_CODE_1__ 本质上是 DAG。

因此本标准使用：

Build Graph gives construction dependencies and scheduling constraints.

而不是错误地把 Build Graph 描述成一个全局线性 construction order。

0.7 6. Fundamental Distinction

整份标准建立在以下区别之上：

因此：

Entity	Primary Responsibility
Declaration	最小命名语义单元
Source File	semantic container / namespace / type locality
Directory	physical organization
Module	compilation dependency boundary
Package	dependency / identity / distribution lifecycle
Artifact	executable / linking / testing product
Build Step	construction dependency / task topology

Logical Architecture MUST NOT be mechanically derived from Physical Structure.

0.8 7. The Eight Architecture Laws

0.8.1 ZIG-ARCH-01 —Source Files Are Semantic Containers

Authority: [O-S] + [E]

Source File 不只是物理文本文件。它 SHOULD 表达：

one coherent type or one coherent namespace

例如：

```
pub const Header = struct {
    // ...
};

pub const Message = union(enum) {
    // ...
};

pub fn encode(...) ... {
    // ...
}

pub fn decode(...) ... {
    // ...
}
```

如果这些 declarations 共同表达一个紧密的 protocol capability,就没有必要机械拆成 __INLINE_CODE_0__, __INLINE_CODE_1__, __INLINE_CODE_2__, __INLINE_CODE_3__, __INLINE_CODE_4__。只有当某个对象已经产生足够强的独立语义压力时,才 SHOULD 拆分 Source File。

0.8.2 ZIG-ARCH-02 —Filesystem Hierarchy Is Not the Dependency Graph

Authority: [O-S] + [E]

例如：

```
src/
├─ config/
├─ protocol/
├─ runtime/
└─ storage/
```

绝不自动意味着存在 __INLINE_CODE_0__, __INLINE_CODE_1__, __INLINE_CODE_2__, __INLINE_CODE_3__,

Directory 提供 Physical Organization。Module 提供 Compilation Dependency。
因此：

Filesystem hierarchy MUST NOT be treated as the Module Graph.

0.8.3 ZIG-ARCH-03 —Modules Are Explicit Compilation Boundaries

Authority: [O-S] + [E]

真正的跨子系统 dependency SHOULD 通过 Module Graph 显式表达。
Module SHOULD 在出现以下 Architecture Driver 时建立：

- independent public surface；
- cross-subsystem dependency；
- dependency isolation；
- different compilation configuration；
- shared use by multiple artifacts；
- stable subsystem boundary；
- architecture topology 需要在 `__INLINE_CODE_0__` 中显式表达。

Module MUST NOT 因为“这里有个目录”而自动创建。
同时必须注意：

Module 是 dependency boundary，不是安全边界，也不是传统 OOP 意义上的 private namespace。

不要把 Module 神化成 Zig 并未提供的访问控制系统。

0.8.4 ZIG-ARCH-04 —Packages Represent Independent Lifecycle Boundaries

Authority: [O-S foundation] + [E]

Package SHOULD 在组件真正出现以下压力时建立：

```
Independent Dependency Lifecycle
+ Independent Identity
+ Independent Distribution
+ Independent Version / Release Concern
+ Independent Build Ownership
```

并不是每一个 Module 都值得成为 Package。
因此：

one package + multiple modules + multiple artifacts

完全可以是一个成熟大型 Zig 项目的正确形态。

0.8.5 ZIG-ARCH-05 —Artifacts Are Orthogonal to Modules

Authority: [O-S] + [E]

Module 回答: **What code depends on what code?**

Artifact 回答: **What gets built?**

例如一个 `__INLINE_CODE_0__` 可以同时服务 `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, `__INLINE_CODE_5__`。

因此:

Module Graph MUST NOT be derived from Artifact Graph, and Artifact Graph MUST NOT be derived from directory structure.

0.8.6 ZIG-ARCH-06 —Discovery Must Be Intentional

Authority: [O-S] + [E]

项目 MUST 明确理解:

repository membership $\neq$ compilation membership

同样:

test source exists $\neq$ test is discovered $\neq$ test artifact is executed

Source / Test membership SHOULD 可以从 import graph、root source files、test roots、Build Graph 清晰推理出来。

0.8.7 ZIG-ARCH-07 —Structure Follows Semantic Pressure

Authority: [E]

本标准既不追求 **Flatness**, 也不崇拜 **Layering**。正确目标是:

the minimum structural depth required to express real semantic boundaries

因此:

```
compiler/
├── frontend/
│   ├── lexer/
│   └── parser/
│       └── ast/
├── semantic/
├── backend/
└── linker/
```

只要每一层都表达真实语义, 就是合理结构。应该消灭的是 **Artificial Hierarchy**, 而不是 **Hierarchy itself**。

0.8.8 ZIG-ARCH-08 —Naming Must Reveal Semantics

Authority: [O-G] + [E]

Zig 0.16 Style Guide 明确警告在 type names 中使用过度泛化的 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, `__INLINE_CODE_5__`, `__INLINE_CODE_6__`。因为这些词往往没有提供真正的分类信息。

本标准将这一原则扩展到 Architecture Naming。以下名称 SHOULD 被视为 semantic smell:

```
utils.zig  
misc.zig  
helpers.zig  
common.zig  
managers/  
contexts/
```

除非项目能够清楚解释其真正语义。例如 `__INLINE_CODE_0__` 更可能应该成为 `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`。
命名本身就是 Architecture Design。

0.9 8. Boundary Decision Framework

结构不能按照“项目越来越大”机械升级。正确方式是根据 boundary pressure 作决定。

0.9.1 8.1 When to Create a New File?

创建新 Source File SHOULD 至少满足一项：

- 一个 coherent type 已经足够 substantial；
- 一个 namespace 已经形成独立语义；
- 当前 file 混合了多个不相关 responsibilities；
- 拆分可以明显改善 locality；
- 拆分可以形成更清晰的 semantic container。

不要默认 `__INLINE_CODE_0__`，也不要无限维持 `__INLINE_CODE_1__`。

0.9.2 8.2 When to Create a Directory?

Directory SHOULD 解决：

- physical navigation；
- semantic grouping；
- large subsystem organization；
- platform/backend grouping；
- fixtures/assets 等资源定位。

Directory 本身 **SHOULD NOT** 被认为产生 Module、Package、Public API、Artifact。创建 Directory 的理由可以只是：“这些文件在语义和认知上属于一起。”这已经足够。

0.9.3 8.3 When to Create a Module?

创建新 Module SHOULD 至少存在一个明确 driver:

- independent dependency boundary;
- independent public surface;
- dependency isolation;
- compilation configuration difference;
- multiple artifact reuse;
- architectural dependency direction;
- stable subsystem contract。

如果唯一理由是“这里已经有一个目录”，则 **SHOULD NOT** 创建 Module。

0.9.4 8.4 When to Create a Package?

Package SHOULD 解决:

- dependency lifecycle;
- package identity;
- external consumption;
- independent distribution;
- independent release/version concern;
- independent ownership。

如果一个 component 永远和主项目一起发布、使用相同 dependency policy、没有 external consumer、没有独立 ownership、不需要独立 distribution, 那么它大概率只是 **Module or internal source subsystem** 而不是 Package。

0.9.5 8.5 When to Create an Artifact?

Artifact SHOULD 对应一个真正 Build Product:

```
CLI executable
Daemon executable
Static library
Dynamic library
Test artifact
Object artifact
Code generator executable
```

不要为了某个代码目录创建 Artifact。Artifact 是 **Runtime / Linking / Testing Boundary**, 不是 **Organizational Folder Boundary**。

0.9.6 8.6 When to Create a Build Step?

Build Step SHOULD 表达一个真正的 construction dependency，例如：

- compile;
- install;
- run;
- test;
- code generation;
- asset generation;
- packaging;
- verification;
- invocation of finite build tools。

不要为了“视觉上有阶段”而人为制造没有 dependency 意义的 Steps。

0.10 9. Source Organization Standard

0.10.1 9.1 File Naming

Authority: [O-G]

Zig Style Guide 将 source files 分成 type-like 与 namespace-like 两类：

- 文件拥有 top-level fields、表达可实例化 type 时，SHOULD 使用 `__INLINE_CODE_0__`；
- 否则作为 namespace-like file，SHOULD 使用 `__INLINE_CODE_0__`；
- directories SHOULD 使用 `__INLINE_CODE_0__`。

官方同时明确指出这些是 general rules of thumb，有合理理由时可以偏离。

例如：

```
Client.zig
Parser.zig
Config.zig

protocol.zig
checksum.zig
process.zig
```

0.10.2 9.2 File-as-Type

File-as-Type 是 Zig source-file struct 语义的自然使用方式，不是模仿 class。

例如：

```
buffer: []const u8,
position: usize,

const Parser = @This();

pub fn init(buffer: []const u8) Parser {
    return .{
        .buffer = buffer,
        .position = 0,
    };
}
```

然后：

```
const Parser = @import("Parser.zig");
```

这直接利用 Source File Struct，而不是人为模拟 OOP class。

0.11 10. Import Architecture

0.11.1 10.1 Local Implementation Relationship

Authority: [E]

同一 Module 内部的 implementation composition MAY 使用相对 source-file import：

```
const parser = @import("parser.zig");
const codec = @import("internal/codec.zig");
```

它表达 **local implementation relationship**。

0.11.2 10.2 Cross-Boundary Relationship

跨稳定 Architecture Boundary 时 SHOULD 优先使用 Named Module：

```
const protocol = @import("protocol");
```

表达 **explicit compilation dependency**。

因此应高度警惕：

```
@import("../common/protocol.zig");
```

如果这条路径穿越了真实 architecture boundary，它意味着 **Filesystem Navigation is encoding Architecture Dependency**。

本标准规定：

Relative imports SHOULD remain within a coherent local implementation boundary. Cross-subsystem dependencies SHOULD normally be represented as named Module dependencies.

这是 `__INLINE_CODE_0__`，而不是 Zig 编译器的普遍代码风格要求。

0.12 11. `__INLINE_CODE_0__` Standard

0.12.1 11.1 Root Source File $\neq$ `__INLINE_CODE_0__`

Authority: [O-S] + [P]

必须严格区分 `__INLINE_CODE_0__` 与 `__INLINE_CODE_1__`。

Zig Compilation 中存在特殊 root module, 且 `__INLINE_CODE_0__` 指向它的 root source file; 这个文件并不必须叫 `__INLINE_CODE_1__`。

因此:

`__INLINE_CODE_0__` has no intrinsic language magic merely because of its filename.

当前官方 `__INLINE_CODE_0__` 会生成 `__INLINE_CODE_1__` 和 `__INLINE_CODE_2__`, 说明这种 topology 是官方脚手架展示的自然默认之一, 但不是语言要求。

0.12.2 11.2 Module Root as Intentional Public Surface

Authority: [E]

当项目选择 `__INLINE_CODE_0__` 作为 reusable Module root 时, SHOULD 将它视为:

Module Facade / Intentional Public Surface

例如:

```
pub const Client = @import("Client.zig");
pub const Config = @import("Config.zig");

pub const protocol = @import("protocol.zig");

pub const Error = error{
    InvalidConfiguration,
    ConnectionFailed,
};
```

内部 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__` 不需要机械 re-export。

Module Root SHOULD 回答: 外部 **consumer** 被允许长期依赖什么? 而不是: 当前目录里有哪些文件?

因此:

A Module Root SHOULD expose a domain contract, not a source-file inventory.

0.13 12. Process Boundary Standard

0.13.1 12.1 Thin `__INLINE_CODE_0__`

Authority: [E]

对于 Application / CLI:

`__INLINE_CODE_0__` SHOULD represent the process boundary.

它通常负责:

- process entry;
- process capabilities;
- argument/environment acquisition;
- top-level I/O capability;

- allocator/resource boundary;
- fatal-error translation;
- exit semantics;
- translation from ambient process state into explicit application inputs.

它 **SHOULD NOT** 成为 CLI parsing + configuration engine + networking + storage + retry policy + protocol implementation + state machine + business workflow 的总容器。

0.13.2 12.2 Zig 0.16 Process Capability Model

Zig 0.16 引入 `__INLINE_CODE_0__` 作为 `__INLINE_CODE_1__` 的可选参数，为程序提供预初始化的 allocator、arena、I/O、environment map、argv 等 process-level capabilities。

因此，本标准将传统 `__INLINE_CODE_0__` 提升为更准确的：

`__INLINE_CODE_0__` owns the process capability boundary.

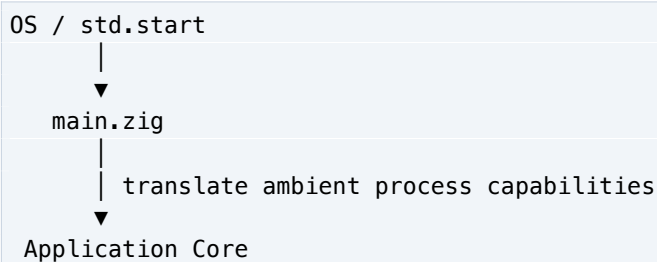
典型形态：

```
const std = @import("std");
const app = @import("app");

pub fn main(init: std.process.Init) !void {
    const args =
        try init.minimal.args.toSlice(init.arena.allocator());

    try app.run(
        init.gpa,
        init.io,
        args,
    );
}
```

核心不是特定函数签名，而是 architecture direction：



因此：

Application Core SHOULD receive explicit capabilities rather than rediscover ambient process state throughout the codebase.

0.14 13. Public API Standard

Public API 是 Architecture Asset。Module Root SHOULD 公开 **semantic contract**，而不是 **physical implementation layout**。

警惕：

```
pub const parser = @import("parser.zig");
pub const lexer = @import("lexer.zig");
pub const cache = @import("cache.zig");
pub const wire = @import("wire.zig");
pub const internal = @import("internal.zig");
```

如果 consumer 最终依赖 `__INLINE_CODE_0__`，那么 physical refactor 将演变成 API break。
更稳定的 surface：

```
pub const Client = @import("Client.zig");
pub const Config = @import("Config.zig");
pub const Request = @import("request.zig").Request;
```

Conformance test:

Can the implementation directory be reorganized without forcing consumers to change imports?

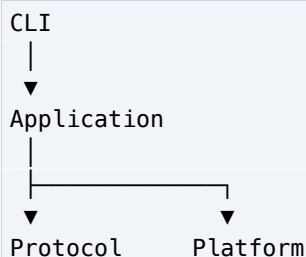
如果答案始终是否定的，Public Boundary 可能尚未形成。

0.15 14. Dependency Architecture Standard

0.15.1 14.1 Dependency Direction Must Be Explainable

成熟项目 SHOULD 能够画出 Module Dependency Graph。

例如：



Architecture Review MUST 能回答：哪个 **Module** 被允许依赖哪个 **Module**？，而不是：“反正文件能 import 到就行。”

0.15.2 14.2 Dependency Cycles

Authority: [O-S] + [E]

Zig 官方明确允许 Module dependency loops。但本标准规定：

Semantic ownership cycles SHOULD be avoided by default.

如果出现 `__INLINE_CODE_0__`，第一步 SHOULD 检查：

- ownership 是否混乱；
- API 是否双向泄漏；
- 是否缺少共同 semantic abstraction；
- 是否应该重新划分 subsystem boundary。

可能的重构：



但：

不要机械地为了消灭 graph cycle 而制造毫无语义价值的 `__INLINE_CODE_0__ Module`。

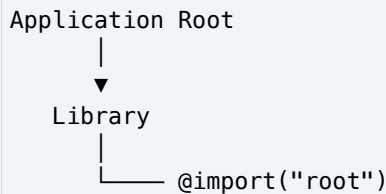
如果所谓 `__INLINE_CODE_0__` 只是垃圾桶 (`__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`)，那通常只是把问题移动了位置。

0.15.3 14.3 Deliberate Root Backreferences

root module 被隐式暴露为 `__INLINE_CODE_0__`，官方也描述了 library 读取 root declarations 以获取应用级全局配置的模式。

因此必须区分 `__INLINE_CODE_0__` 与 `__INLINE_CODE_1__`。

例如：



这种关系 MAY 是有意设计的 host-provided configuration。

所以：

Do not mechanically equate every graph backreference with broken ownership.

真正需要审查的是：**semantic ownership 是否循环。**

0.16 15. Build Architecture Standard

0.16.1 15.1 `__INLINE_CODE_0__` Owns Build Topology

Authority: [O-S] + [E]

Zig Build System 可以声明和配置 build artifacts、options、tests、generated files、system/project tools 等，并以 Step DAG 组织任务。

因此 `__INLINE_CODE_0__` SHOULD 拥有：

- Modules；
- module dependency wiring；
- Artifacts；
- targets；
- optimization configuration；
- package dependencies；
- build options；

- test compilations;
- run steps;
- code generation;
- packaging-related tasks;
- build dependency topology。

核心原则：

__INLINE_CODE_0__ owns construction topology, not application semantics.

0.16.2 15.2 Permitted Build Work

__INLINE_CODE_0__ MAY 编排 finite、reproducible、build-related 的操作，例如：

compile, generate, verify, package, run project tool, run system tool, install, test

官方 Build System 本身支持 generated files、system tools 与 project tools。因此本标准不是：“任何外部命令都不能放进 build.zig。”

0.16.3 15.3 Runtime Boundary

__INLINE_CODE_0__ **SHOULD NOT** 成为以下行为的长期所有者：

- application runtime state;
- business workflow;
- production reconciliation;
- long-lived deployment control plane;
- runtime recovery policy;
- operational state machine。

例如：

```
desired production state
  ↓
continuous reconciliation
  ↓
failure recovery
```

不属于 __INLINE_CODE_0__。即使 Zig Build API 在技术上可以启动某些程序，也不意味着 Build System 应该拥有其运行时语义。

0.17 16. Test Architecture Standard

0.17.1 16.1 Unit Tests Near Semantics

Authority: [E]

默认情况下 Unit Test SHOULD 与被测试 declaration 共置。
例如：

```
fn decodeFrame(...) !Frame {
    // ...
}

test "decodeFrame rejects truncated header" {
    // ...
}
```

因为 unit test 测试的是 **Local Semantic Contract**，应尽可能保持 `__INLINE_CODE_0__` 的 locality。

0.17.2 16.2 Integration Tests Use Explicit Test Roots

Integration / System Tests SHOULD 使用独立 test root，例如：

```
tests/
├─ root.zig
├─ integration.zig
└─ fixtures/
```

然后由 Build System 显式创建相应 Test Compilation。

0.17.3 16.3 Test Discovery Must Be Explicit

Zig test discovery 依赖 Compilation Discovery，而不是递归扫描 `__INLINE_CODE_0__`。官方给出的常见 discovery strategy 包括在 `__INLINE_CODE_1__` / `__INLINE_CODE_2__` block 中显式 `__INLINE_CODE_3__` 需要发现的源码。

例如：

```
test {
    _ = @import("integration.zig");
}
```

因此：

Test membership SHOULD be visible from the import/discovery graph.

0.17.4 16.4 Dependency Module Tests Are Not Automatically the Root Suite

0.16 discovery rules 对 test declarations 的自动发现明确关联：**type/file 是否位于该 test compilation 的 root module。**

因此多 Module 项目 SHOULD 显式决定 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__` 分别属于哪个 Test Compilation。不要假设：“依赖 Module 里写了 test，所以 root test suite 一定自动运行它们。”

0.17.5 16.5 Test Compile ≠ Test Run

使用 Zig Build System 时, test compilation 与 test execution 是不同 Build Steps。官方明确指出: 如果没有通过 `__INLINE_CODE_0__` 等方式建立 Run Step dependency, 仅创建 test compilation 并不会执行 tests。

因此:

Test Exists → Discovered → Compiled → Run Step Exists → Executed

这些状态 MUST 被区分。

0.17.6 16.6 `__INLINE_CODE_0__` Is Not the Architecture Baseline

Authority: [E]

本标准不规定 `__INLINE_CODE_0__` 作为项目标准 test membership mechanism。

默认策略是让 membership 通过 explicit test roots、`__INLINE_CODE_0__`、discovery graph、Build Graph 清晰可推理。

目标是:

Test topology should be explicit rather than recovered from broad reflection tricks.

特定项目 MAY 使用 `__INLINE_CODE_0__`, 但必须知道自己希望解决的问题是什么。

0.18 17. Naming and Semantic Cohesion Standard

0.18.1 17.1 Generic Names Require Evidence

名称 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, `__INLINE_CODE_5__`, `__INLINE_CODE_6__`, `__INLINE_CODE_7__`, `__INLINE_CODE_8__` 不自动禁止。

但使用它们时 SHOULD 能回答:

What state? What context? What is actually managed? What is actually common?

例如 `__INLINE_CODE_0__` 可能真正意味着 `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, `__INLINE_CODE_5__`; 而 `__INLINE_CODE_6__` 可能真正意味着 `__INLINE_CODE_7__`, `__INLINE_CODE_8__`, `__INLINE_CODE_9__`, `__INLINE_CODE_10__`, `__INLINE_CODE_11__`, `__INLINE_CODE_12__`。命名越准确, Architecture Ownership 越清晰。

0.18.2 17.2 Avoid Ceremonial Layers

本标准不禁止 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, 也不默认推荐它们。如果问题本身确实具有这些变化边界和 dependency direction, 它们完全合理。

但不要为了模仿 Clean Architecture, Hexagonal Architecture, DDD 而预先生成 `__INLINE_CODE_0__`, `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__`, `__INLINE_CODE_5__`。Architecture Boundary SHOULD 至少表达:

- independent semantics;
- independent dependency pressure;
- independent reason to change.

否则它只是 **Ceremonial Architecture**。

0.19 18. Evolution Standard —Structure Must Be Earned

项目 SHOULD 按真实 complexity 演化：

```
Single File
↓
Several Semantic Files
↓
Intentional Public Surface
↓
Multiple Modules
↓
Multiple Artifacts
↓
Multiple Packages
```

这是一种可能的 evolution path，不是一条必须逐级经过的状态机。禁止：

```
New Project → Generate Enterprise Architecture
```

抽象必须通过真实复杂性获得存在资格。

0.20 19. Reference Pattern A —Minimal Program

Authority: [P]

真正简单时：

```
hello/
└─ main.zig
```

已经足够。甚至不一定需要 Build System。不要为了显得专业，从第一天创建 `__INLINE_CODE_0__`，`__INLINE_CODE_1__`，`__INLINE_CODE_2__`，`__INLINE_CODE_3__`，`__INLINE_CODE_4__`。

0.21 20. Reference Pattern B —Small Application

Authority: [P]

随着第一个真实 semantic boundary 出现：

```
project/
├─ build.zig
├─ build.zig.zon
└─ src/
   ├─ main.zig
   ├─ config.zig
   └─ process.zig
```

仍然完全可以只有一个 project Module。

0.22 21. Reference Pattern C —Medium Application

Authority: [P]

当核心逻辑需要与 process boundary 分离：

```
project/
├─ build.zig
├─ build.zig.zon
└─
```

```

├── src/
│   ├── main.zig
│   ├── root.zig
│   ├── Config.zig
│   ├── Engine.zig
│   ├── protocol.zig
│   ├── process.zig
│   └── platform/
│       ├── posix.zig
│       └── windows.zig
└── tests/
    └── root.zig

```

可能的 logical relationship:

```

main.zig
  ↓
Application Module
  ↓
root.zig facade
  ├── Config
  ├── Engine
  ├── protocol
  └── implementation files

```

这里所有文件名和目录都是 **Good Defaults**, 不是 Zig language requirements。

0.23 22. Reference Pattern D —Multi-Artifact System

Authority: [P]

当同一个 Core 服务多个独立 executable:

```

project/
├── build.zig
├── build.zig.zon
└── src/
    ├── core/
    │   ├── root.zig
    │   ├── Engine.zig
    │   └── protocol.zig
    ├── cli/
    │   └── main.zig
    └── daemon/
        └── main.zig

```

Module Graph:

```

core

```



Artifact Graph:

```

core
├── CLI executable
├── daemon executable
└── test artifacts
  
```

Physical Tree、Module Graph、Artifact Graph 明确不同。

0.24 23. Reference Pattern E —Multi-Package Repository

Authority: [P]

只有出现真正 independent lifecycle 时再考虑：

```

project/
├── packages/
│   ├── protocol/
│   │   ├── build.zig
│   │   └── build.zig.zon
│   └── engine/
│       ├── build.zig
│       └── build.zig.zon
└── apps/
    └── cli/
  
```

不要因为 `__INLINE_CODE_0__` “看起来很独立”，就立即升级为 Package。Package Boundary 是 **lifecycle decision**，而不是 **folder size decision**。

0.25 24. Consolidated Anti-Patterns

以下模式默认视为 Architecture Smell。

0.25.1 AP-01 —Filesystem as Dependency Injection

```
@import("../../../../../foo/bar.zig");
```

跨越真实 architecture boundary 时，用物理路径编码逻辑依赖。

0.25.2 AP-02 —Module per Folder

folder → module

机械一一对应。这只是重新引入 **Directory-as-Package thinking**。

0.25.3 AP-03 —Package per Module

module → package

忽略两者生命周期语义完全不同。

0.25.4 AP-04 —Fat __INLINE_CODE_0__

Process Entry 直接拥有 storage、networking、retry、protocol、state machine、business semantics。

0.25.5 AP-05 —God __INLINE_CODE_0__

Thin __INLINE_CODE_0__ 不意味着：把所有实现搬进 __INLINE_CODE_1__。正确目标是：

root.zig → Intentional Public Surface → Coherent Implementation

0.25.6 AP-06 —Runtime Control Plane in __INLINE_CODE_0__

长期 operational state、runtime reconciliation 或 recovery policy 被塞进 Build Graph。

0.25.7 AP-07 —Forced Flatness

因为“Zig 喜欢简单”而拒绝所有合理的 subsystem hierarchy。

0.25.8 AP-08 —Ceremonial Architecture

因为某个 Architecture Pattern 流行，就提前创建没有语义压力的抽象层。

0.25.9 AP-09 —Semantic Garbage Buckets

__INLINE_CODE_0__, __INLINE_CODE_1__, __INLINE_CODE_2__, __INLINE_CODE_3__, __INLINE_CODE_4__ 不断吸收无法分类的逻辑。

0.25.10 AP-10 —Premature Shared Module

看到 __INLINE_CODE_0__ 立刻创建 __INLINE_CODE_1__, __INLINE_CODE_2__, 却没有真正发现 shared semantic contract。

0.26 25. Architecture Decision Matrix

Question	Create	Primary Reason
一个 type/namespace 已形成独立 cohesive semantics?	File	Semantic locality
多个 files 需要更好的物理导航?	Directory	Physical organization
出现 稳定 compilation dependency boundary?	Module	Dependency architecture
出现 独立 identity/distribution lifecycle?	Package	Lifecycle boundary
出现 独立 executable/link/test product?	Artifact	Product boundary
构建任务之间出现明确 dependency constraint?	Build Step	Construction topology

任何 Architecture Review 在新增结构前 SHOULD 先回答：

What pressure is this boundary responding to?

如果答案只是：“这样看起来更标准”，通常不应创建该 boundary。

0.27 26. Conformance Review

Code Review / Architecture Review 不应该首先问：“这个文件应该放哪个标准目录？”，而应该按以下顺序检查。

0.27.1 Semantic Ownership

1. What semantic responsibility does this declaration own?
2. Does that responsibility deserve a separate file?
3. Is the file a coherent type or namespace?

0.27.2 Dependency

1. Is this relationship local implementation or cross-boundary dependency?
2. Does this boundary deserve a Module?
3. Can the Module Dependency Graph be explained?
4. Is a dependency cycle representing real semantic ownership confusion?

0.27.3 Lifecycle

1. Does this component really need independent Package identity or lifecycle?

0.27.4 Product

1. Is this a Module boundary or an Artifact boundary?

0.27.5 Discovery & Testing

1. Will this source actually enter the Discovery Graph?
2. Will this test actually be discovered?
3. Is its Test Compilation actually executed?

0.27.6 Build

1. Does `__INLINE_CODE_0__` describe construction topology or application runtime behavior?

0.27.7 API

1. Can the public API survive internal directory reorganization?

0.27.8 Naming

1. Does the chosen name reveal semantics or hide missing semantics?

如果这些问题都有清晰答案：目录结构通常会自然变得正确。

0.28 27. Conformance Levels

0.28.1 Required

所有 `__INLINE_CODE_0__` 和 `__INLINE_CODE_1__` 构成本标准的强制 Architecture Contract。

0.28.2 Recommended

所有 `__INLINE_CODE_0__` 和 `__INLINE_CODE_1__` 允许存在有理由的 deviation。但 deviation SHOULD 可以回答：

Why is this exception architecturally better here?

0.28.3 Preference

所有 `__INLINE_CODE_0__` 只是 recommended baseline。例如 `__INLINE_CODE_1__`, `__INLINE_CODE_2__`, `__INLINE_CODE_3__`, `__INLINE_CODE_4__` 均不构成 conformance requirement。

0.29 28. Official Semantic Traceability

这张表是 Architecture Standard 与当前 Zig 官方语义之间的 Traceability Boundary。

0.30 29. Frozen Engineering Decisions

在 Zig 0.16 baseline 下，本标准冻结以下工程结论：

```
Thin main.zig
Intentional Module Root
Relative imports stay local by default
Named Modules express cross-subsystem dependencies
Do not create one Module per directory
Do not create one Package per Module
Unit tests live near local semantics by default
Integration tests use explicit test roots
Test membership must be intentional
Semantic dependency cycles are smells by default
Public API must not mirror file inventory
build.zig owns construction topology
Long-lived runtime semantics stay outside build.zig
Naming must reveal semantic ownership
Structure must be earned by real complexity
```

这些属于 **Architecture choices derived from Zig semantics**，而不是 Zig 语言本身的 universal requirements。

0.31 30. Final Architecture Model

成熟 Zig 项目不应该只展示 `__INLINE_CODE_0__` 目录树。它 SHOULD 能够同时解释：

```
Physical Source Tree
|
Semantic Container / Namespace View
|
Module Dependency Graph
```

ID	Statement	Authority	Official Basis
ZIG-SEM-01	Every source file is implicitly a <code>__INLINE_CODE_0__</code>	O-S	Zig 0.16 Compilation Model
ZIG-SEM-02	Module has a root source file	O-S	Compilation Model
ZIG-SEM-03	Modules can depend on Modules by name	O-S	Compilation Model
ZIG-SEM-04	Module dependency loops are allowed	O-S	Compilation Model
ZIG-SEM-05	<code>__INLINE_CODE_0__</code> refers to root module	O-S	Compilation Model
ZIG-SEM-06	Discovery follows explicit recursive rules	O-S	File and Declaration Discovery
ZIG-SEM-07	Root-module test discovery has special semantics	O-S	File and Declaration Discovery
ZIG-BUILD-01	Build System is modeled as a Step DAG	O-S	Zig Build System
ZIG-BUILD-02	Test Compile and Test Run are distinct Steps	O-S	Zig Build System Testing
ZIG-STYLE-01	File-as-type uses Title-Case by convention	O-G	Style Guide
ZIG-STYLE-02	Namespace-like files/directories use snake_case	O-G	Style Guide
ZIG-STYLE-03	Generic names such as Manager/State/utils/misc are discouraged	O-G	Style Guide
ZIG-PROC-01	<code>__INLINE_CODE_0__</code> exposes process capabilities to <code>__INLINE_CODE_1__</code>	O-S/API	Zig 0.16 Release Notes
ZIG-PKG-01	0.16 package identity uses package metadata including name/fingerprint in override resolution	O-S/tooling	Zig 0.16 Release Notes
ZIG-PREF-01	Current <code>__INLINE_CODE_0__</code> emits <code>__INLINE_CODE_1__</code> and <code>__INLINE_CODE_2__</code>	O-G/example	Official Getting Started

```

|
| Declaration Discovery Graph
|
| Package Dependency / Lifecycle View
|
| Artifact / Product Graph
|
| Build Step DAG

```

这些不是一张图，也不是一个机械层级。它们共同构成项目的 Architecture Model。

0.32 31. Final Manifesto

Zig 项目架构的目标不是创造最漂亮的目录树。

不是最大程度减少文件数量。

不是模仿 Java Package。

不是模仿 Go Package。

不是模仿 Rust Crate。

不是因为 Zig 强调简单就拒绝 Architecture。

也不是因为大型系统复杂，就提前制造 Enterprise Ceremony。

真正应该保持的是：

File gives semantic locality. Directory gives physical organization. Module gives explicit compilation dependencies. Package gives lifecycle identity. Artifact gives build products. Discovery determines compilation reality. Build DAG gives construction dependencies. Naming communicates semantic intent. Explicitness gives architecture clarity.

0.33 32. Architecture Freeze Statement

本 Standard v1.0 最终冻结的不是：

```

src/
├─ main.zig
├─ root.zig
└─ ...

```

这样的目录模板。

它冻结的是以下 Architecture Reasoning Model：

Do not derive logical architecture mechanically from physical structure.

在 Zig 中：

File ≠ Directory ≠ Module ≠ Package ≠ Artifact ≠ Build Step

这些对象拥有不同职责。只有当真实语义产生新的 boundary pressure 时，结构才应该增长。

因此，本标准最终坚持：

Neither flatness nor layering is the Zig architecture ideal. Explicitness is.

项目应该：

Start simple. Discover real boundaries. Make those boundaries explicit. Keep dependency direction explainable. Keep compilation reality observable. Stop abstracting when the semantics are already clear.